

PA 1: Implementing xv6 User Commands and System Calls

Please replace red text with your report text and any tables or figures, names of any accompanying files, etc. Remember to commit all the files for your lab submission, to put the URL for your private xv6 repo in the Teams assignment, to submit the Teams assignment, and to give the instructor and TA access to your repo.

Task 1. (5 pts) Implement the uptime utility
result was 36 and 66 when running it a second time

```
xv6 kernel is booting

init: starting sh
$ uptime
up 36 clock ticks
$ uptime
up 66 clock ticks
$ █
```

Task 2. (10 pts) Implement a time1 command that reports elapsed time.

```
xv6 kernel is booting

init: starting sh
$ time1 matmul
Time: 5 ticks
elapsed time: 5 ticks
$ time1 matmul & time1 matmul &
$ Time: 9 ticks
elapsed time: 10 ticks
Time: 9 ticks
elapsed time: 10 ticks

$ time1 sleep 10
elapsed time: 10 ticks
$ █
```

Task 3. (10 pts) Keep track of how much cputime a process has used.
Proc.h : I added a int cputime field in struct proc; the purpose is to store the total number of timer ticks that the process spends actively executing

Proc.c : Changed for the field to initialize to zero when the process is created in allocproc(); the purpose was to ensure a fresh process starts

Trap.c : In usertrap() and kerneltrap() I changed for the fields to increase every time the process uses up its time slice; the purpose was to ensure we see how much the process spends executing

Task 4. (10 pts) Implement a wait2() system call that waits for a child to exit and returns the child's status and rusage.

Pstat.h : created a new file with the contents provided to add; which purpose is that it defines struct rusage containing the cputime field

user.h : added the contents that I was provided with of "struct rusage" and a system call prototype "int wait2(int*, struct rusage*);"; which allows for struct rusage to be recognized or declared and invoke the wait2() system call

usys.pl : added an entry for wait2(); the purpose is for the assembly code to be created letting the user program trigger the wait2 system call into the kernel.

Syscall.h : added a wait2 system call information or "#define SYS_wait2 22"; the purpose is for it to assign a unique system call number identifying wait2

syscal.c : added a wait2 system call information by declaring "extern uint64 sys_wait2(void);" and registering "[SYS_wait2] sys_wait2,"; which its purpose is to map the system call number to the "sys_wait2" handler

defs.h : added "int kwait2(uint64, uint64);"; the purpose for this is for wait2() function declaration to be exported so it is visible and callable from sysproc.c

sysproc.c : implemented sys_wait2() to return child process status and rusage;; in which purpose is for it to read the system call arguments and pass them to kwait2().

Proc.c : implemented the kwait2() function to the file and added "#include "pstat.h"" also; Which its purpose is that it waits for a child process to exit, grabs its cputime, and copies both exit status and rusage back to the user space.

Task 5. (10 pts) Implement a time command that runs the command given to it as an argument and outputs elapsed time, CPU time, and %CPU used.

```
xv6 kernel is booting

init: starting sh
$ matmul
Time: 5 ticks
$ time matmul
Time: 5 ticks
elapsed time: 5 ticks, cpu time: 5 ticks, 100% CPU
$ time matmul & time matmul &
$ Time: 9 ticks
elapsed time: 10 ticks, cpu time: 5 ticks, 50% CPU
Time: 8 ticks
elapsed time: 10 ticks, cpu time: 6 ticks, 60% CPU

$ time sleep 10
elapsed time: 10 ticks, cpu time: 0 ticks, 0% CPU
$ █
```

Task 6. (5 pts) Summarize what you learned from doing this assignment. Describe any difficulties you faced and if/how you overcame them.

Through this assignment, I managed to learn how the xv6 system calls work like from end to end, which is from within the user to the kernel. Another thing I managed to learn from this assignment was understanding the difference between clock time and actual processor time, by seeing how when multiple programs share a single CPU, will have a different time to finish compared to when only receiving about an equal fraction. Which I was able to do by making a command that measures both the total time the command takes to finish, and the time it spends using the processor.

A primary challenge I would say I faced and was able to overcome was mainly in task 4. Which we were tasked with implementing a wait2() system call and had to change or add across multiple files that will be able to work together. I was able to overcome this by correctly making sure the code across the files manage to work together and carefully going through the existing kwait() function and applying it to my kwait2() logic, so it will be able to match everything like the exact parameters and locking pattern.

Extra Credit (5 points). Discuss limitations of our time command. (Hint: For one limitation, consider what would happen if the command that is being timed forks child processes).